



Data Mining and Machine Learning: Vehicle Insurance Fraud Detection

By Hussein Sinan Nizar

LinkedIn: <https://www.linkedin.com/in/hussein-tamimme-1943072a5/>

Github: <https://github.com/HusseinTamimme>

Introduction:

Vehicle insurance fraud detection is the process of identifying and preventing false or exaggerated claims made by policyholders to gain unwarranted financial benefits. This type of fraud can include staged accidents, false injury claims, or inflating repair costs. Insurance companies use various investigative methods such as claim verification, interviews, examination of evidence, and cross-checking with databases to uncover inconsistencies and suspicious activities. Detecting such fraud is essential to protect the financial stability of insurance providers and to ensure fair premiums for honest customers.

Why Is It Important to Predict Medical Conditions Using Data?

Applying machine learning to vehicle insurance fraud detection is important because it enables faster, more accurate, and scalable identification of fraudulent claims. Unlike traditional manual methods, machine learning can analyze large volumes of complex data, uncover hidden patterns, and detect subtle anomalies that might be missed by human investigators. This helps insurance companies reduce financial losses, improve claim processing efficiency, and focus resources on high-risk cases, ultimately leading to fairer premiums and better customer service.

Libraries used:

```
import pandas as pd
import seaborn as sns
from sklearn.impute import KNNImputer as KNNI
from sklearn.impute import SimpleImputer as SIR
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
from sklearn.ensemble import RandomForestClassifier
from sklearn.feature_selection import SelectFromModel, SelectKBest, chi2
from xgboost import XGBClassifier
from imblearn.combine import SMOTETomek
from imblearn.over_sampling import SMOTE
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, confusion_matrix, roc_curve, auc, classification_report
import matplotlib.pyplot as plt
from imblearn.under_sampling import TomekLinks
```

This is the initial set of libraries and tools that I might use

Pandas: For processing the dataset

Seaborn: For visualization

Sklearn: For machine learning tools

Xgboost: For solving classification and regression problems

Imblearn: tools to fix unbalanced classification problems

Seaborn: Creates informative statistical graphics

Matplotlib: creating a wide variety of visualizations, including static, animated, and interactive plots We will go through each tool in detail when it's used...

Loading the dataset and splitting the features:

Brief Explanation:

Every dataset has Columns and rows (Samples)

There are multiple types for columns; it could be a feature, Features (also called input variables, predictors, or independent variables) are the measurable attributes or properties of the phenomena being observed. They are the variables used by a machine learning model to learn patterns and make predictions.

Another type of column is The Class. In machine learning, especially classification problems, the class (also called the label, output, or target variable) is the variable we want to predict. It represents the category or outcome associated with each observation.

Usually the dataset is splitted into a training set and testing/validating set Where the training set is used to train a machine learning model and the testing set is used to evaluate the model.

In general there are two types of features. Numerical, and Categorical
Numerical features are features where the data in it are numbers
These numbers could be continuous, which means there is an infinite amount of numbers or basically it contains float numbers.

And discrete numbers where the numbers are countable, and usually these numbers are integers.

Now the Categorical features the data are usually textual, there is more details to that but I will get to it later on....

```
dataset=pd.read_csv('Insurance_claims_data_modified.csv')
features= dataset.drop(columns=['claim_status', 'policy_id']).columns.tolist()
numeric_features = dataset[features].select_dtypes(include=['number']).columns.tolist()
categorical_features = dataset[features].select_dtypes(include=['object', 'category']).columns.tolist()
X=dataset[features]
Y=dataset['claim_status']
```

First Line:

Calling the pandas library to load the dataset into the workspace so we can process it.

Second Line:

Extracting the features without the class attribute and dropping irrelevant attributes that won't help in predicting like the

“Policy_id”, in Addition it separates Class attribute; it is useful to do that especially when you want to split the data into a training and testing set.

Third and fourth Line:

Separating features into numerical and categorical; this could help When we start the preprocessing part.

Last 2 lines for Assigning variables for the features and class attribute, X as the features and Y as the class

Now the data is ready to be worked on but before we get to the preprocessing we need to explore the data first to make the right decisions In each preprocessing stage....

Basic Exploration:

```
print(X.shape)
X.describe()
```

(59177, 39)

	subscription_length	vehicle_age	customer_age	region_density	airbags	displacement	cylinder	turning_radius	length	width
count	58601.000000	58602.000000	58581.000000	58619.000000	58554.000000	58627.000000	58610.000000	58601.000000	58590.000000	58590.000000
mean	6.111616	1.389413	44.826241	18818.426022	3.137326	1162.429154	3.626941	4.853003	3850.596126	1672.252842
std	4.141393	1.134188	6.935619	17660.739644	1.832875	266.247469	0.483622	0.228099	311.395012	112.066723
min	0.000000	0.000000	35.000000	290.000000	1.000000	796.000000	3.000000	4.500000	3445.000000	1475.000000
25%	2.100000	0.400000	39.000000	6112.000000	2.000000	796.000000	3.000000	4.600000	3445.000000	1515.000000
50%	5.700000	1.200000	44.000000	8794.000000	2.000000	1197.000000	4.000000	4.800000	3845.000000	1735.000000
75%	10.400000	2.200000	49.000000	27003.000000	6.000000	1493.000000	4.000000	5.000000	3995.000000	1755.000000
max	14.000000	20.000000	75.000000	73430.000000	6.000000	1498.000000	4.000000	5.200000	4300.000000	1811.000000

gross_weight **ncap_rating**

58587.000000 58591.000000

1385.293444 1.759741

Now `.shape` shows how many rows and how many columns are in the dataset, so this shows that there is 59177 rows and 39 features

In the second line `X.describe()` it shows a brief statistical parameter for the numerical features of the dataset.

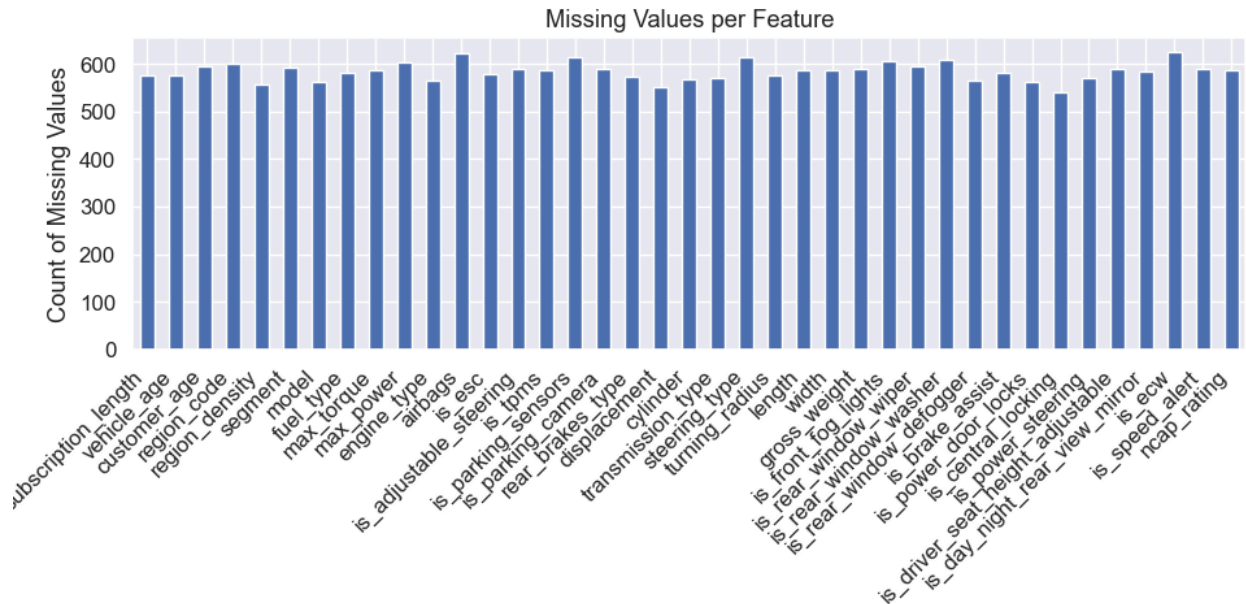
What does all these numbers mean? Count shows how many values are in a certain feature. Mean shows the average of a feature. Std shows the standard deviation of each feature, The standard deviation is a statistical measure that indicates the spread or dispersion of a dataset around its mean (average). Min shows the minimum value in each feature, while max does the opposite. Now these percentages shows the quantiles

For example 25% means that the following value is larger than 25% of the values in a certain feature. But why do we need this information? To consider if our data needs a certain transformation or scaling...

```
print(X.isnull().sum())
```

This statement shows how many null/missing values are in each feature, this helps handling missing values because there are multiple approaches to handle these missing values.

Now to make things more simple I created a bar chart because there are too many columns and a bar chart would be easier to understand



The average of missing values is 584.46 which is really small compared to the number of rows

```

null_counts = null_counts[null_counts > 0]
null_counts.plot(kind='bar', figsize=(10, 5), title='Missing Values per Feature')
plt.ylabel('Count of Missing Values')
plt.xticks(rotation=45, ha='right')
plt.tight_layout()
plt.show()

```

The code Above is responsible for drawing the bar chart

Now in the following image the first line shows if there are any missing values in the class feature. It appears that there are 608 missing values.

In the second line I fix this problem by imputing the mode or the most occurring class. Now this won't be really accurate because it's an imbalance problem but considering the small amount of missing values it won't really affect that much. In the third line we check if there are any missing values but it just showed 0 so it's fixed. The last line shows the number of times each class occurred and you can notice that there is a REALLY significant difference which we will fix later on...

The code below visualizes the significant imbalance:

```

print(Y.isnull().sum())
Y = Y.fillna(Y.mode()[0])
print(Y.isnull().sum())
print(Y.value_counts())

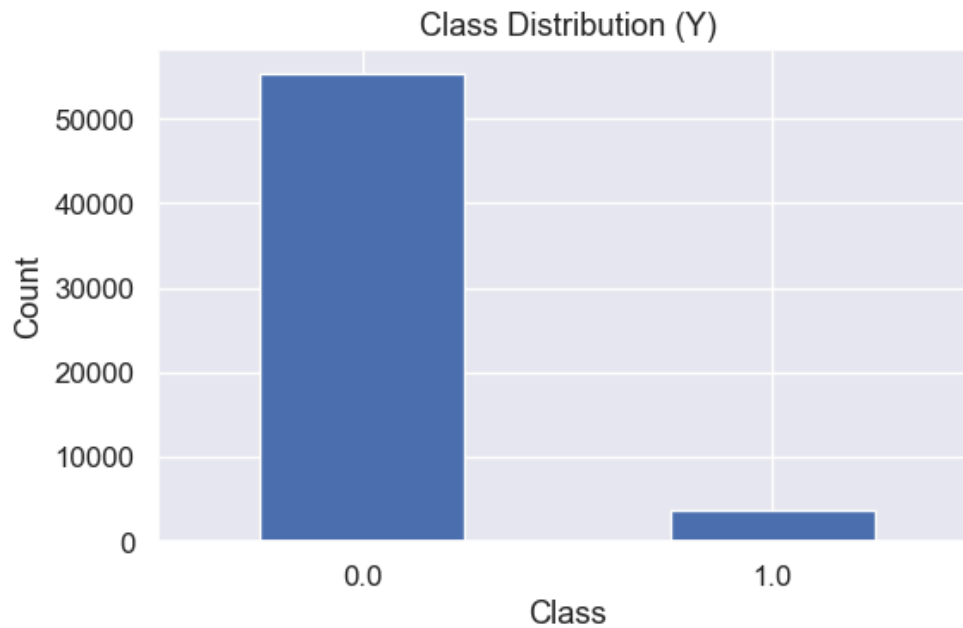
```

```

608
0
claim_status
0.0    55430
1.0     3747
Name: count, dtype: int64

```

```
class_counts = Y.value_counts()
class_counts.plot(kind='bar', figsize=(6, 4), title='Class Distribution (Y)')
plt.ylabel('Count')
plt.xlabel('Class')
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```



Handling missing values:

Most of the Machine learning libraries will give you an error when trying to build a model with missing values.

Now there are multiple approaches:

The first approach is to basically choose a model that can ignore the missing values automatically, but this approach is not really an optimal solution or even the best one.(Acceptance)

The second approach is to drop columns that have a large amount of missing values but here is another thing to consider: if the column or feature is important, this approach won't do any good.

The third approach is to impute, filling values may not be 100% right but it can improve the model accuracy in a way better than dropping columns.

Fourth approach is to drop the rows with missing values, but that won't work here because almost half of the dataset has missing values.

But which one of these approaches should I choose for my dataset?

The first one is not optimal, the second approach can't be applied because all the features included have missing values. This leaves no choice but to impute the values or use the fourth approach but imputing could be better for accuracy so I will go with the Third approach.

Now let's impute the values but what value will we impute exactly? Basically we could use any measure of tendency (e.g mean, median, mode) or use KNN

Which is a distance based algorithm that puts relations into consideration even if the correlations are almost very weak but it's still a better choice than just filling a measure of tendency values.

Now KNN is used for numerical data, For categorical data it's different There are no measures of tendency except the mode(Most frequent), And KNN is used for Numerical data. So Mode is the perfect choice For categorical features.

```
X_train, X_val, y_train, y_val = train_test_split(X, Y, stratify=Y)
print(X_train.shape[0])
```

44382

One last thing before we start the preprocessing steps is we need to train_test_split the data, and that's the purpose of the first line.

By default it will split the rows randomly into 44382 out of 59177 rows which is 75% into the training set and the rest will be for testing as shown using the print statement in the second line.

There is one last thing and that is the “Stratify” basically to prevent the model bias the splitting algorithm Stratifies which means it groups the classes in an equal way so that each group has the same amount of both classes.

```
MyNImputer = KNNI(n_neighbors=7)
ImputedN_X_train = pd.DataFrame(MyNImputer.fit_transform(X_train[numeric_features]),
                                columns=numeric_features, index=X_train.index)
ImputedN_X_val = pd.DataFrame(MyNImputer.transform(X_val[numeric_features]),
                              columns=numeric_features, index=X_val.index)
```

These Lines of code apply the KNN Imputer and it will operate based on 7 neighbors which means for every missing value it will try to impute values based on the seven most similar cases to it.

So KNNI() creates an object of the KNN Imputer then .fit_transform means that “Let the KNN Imputer learn using the data then apply the label encoding on the data” this is applied only on the training set on the testing set you only transform which means that the KNN Imputer only applies KNN Imputing because it learned already.

The KNN will only be applied on numerical features. Now there is one problem sometimes when applying preprocessing techniques the attribute names might disappear so after using some of the preprocessing techniques you have to write columns=... this will assign back the attribute names, it's important to do that because the numerical part of our data is separated from the rest of the data

```
MyCImputer = SIR(strategy='most_frequent')
ImputedC_X_train = pd.DataFrame(MyCImputer.fit_transform(X_train[categorical_features]),
                                columns=categorical_features, index=X_train.index)
ImputedC_X_val = pd.DataFrame(MyCImputer.transform(X_val[categorical_features]),
                              columns=categorical_features, index=X_val.index)
```

This is almost the same as imputing for numerical values but now the difference is this one uses simple imputer because its imputation for

categorical attributes and don't get me wrong Simple Imputer isn't only meant for categorical attributes but it has an option called 'most_frequent' Which can be applied on the categorical attributes. What it does basically it takes the most occurred value and impute it

Normalizing numerical features:

Normalizing is scaling numerical features so that they have a similar range (usually between 0 and 1). This ensures that no single feature dominates others due to its scale. For example the customer ages could be between 20-80 but the ncap_rating is between 0 and 5, they have different ranges and some models may be biased towards higher-magnitude features.

Most common types are the standard scaler, min-max scaler, and robust scaler, but I will use Min Max Scaler and the reason is for its range between 0-1 where 1 means the maximum value appeared and 0 means the opposite. I don't need any negative values since all of my numerical values are positive values only.

```
scaler = MinMaxScaler()
ScaledN_X_train = pd.DataFrame(scaler.fit_transform(ImputedN_X_train),
                               columns=numeric_features, index=X_train.index)
ScaledN_X_val = pd.DataFrame(scaler.transform(ImputedN_X_val),
                              columns=numeric_features, index=X_val.index)
```

I think this one explains itself I will only add that the equation is $(\text{Value} - \text{minimum value}) / (\text{maximum value} - \text{minimum value})$.

Encoding Categorical Features:

There are two levels of measurement for the categorical variables.

First level is the nominal where the categorical values are only different, And there's no exact way or enough information to order these values, it's only different(Distinct).

The second level is Ordinal. The values can be ordered but there's not much information about the ordering.

For example (Low, medium, high) you have enough information just to order them but if it was a numbers you can do much more than that, but We're talking about categorical variables.

Some models don't work well, some of them don't work at all when the dataset has categorical features. So we need to fix this issue by dropping categorical features, but we have 27 categorical features out of 39. So we go to the other option which is something called encoding. Every level of measurement needs a certain type of Encoding, if our data is ordinal we use a method called Direct Conversion or Label Encoder. If the data is nominal we use One Hot

Encoding. I isolated the features into numerical features which we already used and nominal categorical features, binary categorical features based on the values of each categorical features

Which you can see in the following image....

There are certainly more categorical features, but most of the remaining feature's values are "Yes" or "No", feel free to go to the py file and have a look. So it's time to check and filter them based on their values

Any "Yes" or "No" feature is considered binary and can be considered Nominal but in this dataset "Yes" is certainly better than "No" so it can be ordinal. The rest of the features some of them can be considered Ordinal

```
for x in categorical_features:
    print(x)
    print(X[x].unique())
    print("-----")
```

```
region_code
['C8' 'C2' 'C10' 'C13' 'C7' 'C5' 'C3' 'C19' 'C9' 'C15' 'C6' 'C11' 'C1'
 'C14' 'C17' nan 'C12' 'C4' 'C21' 'C16' 'C18' 'C22' 'C20']
-----
segment
['C2' 'C1' 'A' 'B2' 'B1' 'Utility' nan]
-----
model
['M4' 'M9' 'M1' 'M5' 'M7' 'M6' 'M8' nan 'M3' 'M2' 'M10' 'M11']
-----
fuel_type
['Diesel' 'CNG' 'Petrol' nan]
-----
max_torque
['250Nm@2750rpm' '200Nm@1750rpm' '60Nm@3500rpm' '200Nm@3000rpm'
 '113Nm@4400rpm' '82.1Nm@3400rpm' '91Nm@4250rpm' '85Nm@3000rpm' nan
 '170Nm@4000rpm']
-----
max_power
['113.45bhp@4000rpm' '97.89bhp@3600rpm' '40.36bhp@6000rpm'
 '88.77bhp@4000rpm' '88.50bhp@6000rpm' '55.92bhp@5300rpm'
 '67.06bhp@5500rpm' '61.68bhp@6000rpm' '118.36bhp@5500rpm' nan]
-----
engine_type
['1.5 L U2 CRDi' 'i-DTEC' 'F8D Petrol Engine' '1.5 Turbocharged Revotorq'
 '1.2 L K Series Engine' 'K Series Dual jet' 'K10C' '1.0 SCe' nan
 '1.2 L K12N Dualjet' 'G12B' '1.5 Turbocharged Revotron']
-----
```

Like “model” but it doesn’t have an universal way of ordering so I will consider it as nominal. The following code shows the isolation of nominal features and the binary features..

```
NomC_features=[
    'region_code','segment','model','fuel_type','engine_type','rear_brakes_type','transmission_type','steering_type','max_power','max_torque']
binary_features = ['is_esc','is_adjustable_steering','is_tpms','is_parking_sensors','is_parking_camera','is_front_fog_lights',
    'is_rear_window_wiper','is_rear_window_washer','is_rear_window_defogger','is_brake_assist','is_power_door_locks',
    'is_central_locking','is_power_steering','is_driver_seat_height_adjustable','is_day_night_rear_view_mirror',
    'is_ecw','is_speed_alert']

NomEncoded_X_train = pd.get_dummies(ImputedC_X_train[NomC_features], drop_first=True)
NomEncoded_X_val = pd.get_dummies(ImputedC_X_val[NomC_features], drop_first=True)
```

The last two lines apply one hot encoding to nominal features in the training and testing set. The last 2 lines will create a column for each feature and there possible values, for example:

['model_M1','model_M2'] and their values will be true and false

Based on the original feature so if M1 appeared in the original feature model_m1 will have a true value and the rest of the copies will have a false values

But do the original features stay? No because of the drop_first=True argument so all the original features that got encoded will be thrown out.

```
for col in binary_features:
    BinImputedC_X_train = ImputedC_X_train[col].map({'Yes': 1, 'No': 0})
    BinImputedC_X_val = ImputedC_X_val[col].map({'Yes': 1, 'No': 0})
```

This code replaces every “Yes” with 1 and “No” with 0 using mapping.

Didn't we forget something? Did we encode numerical features a moment ago? I mean the max_power and max_torque because there were numerical values inside them, but the reality is these are categorical features because we can’t apply any mathematical operations on them.

Final touch before finishing preprocessing:

After all that encoding and imputing, our data is splitted and isolated into multiple partitions and parts. So we need to combine it back using the first 2 lines in the following code...

```
Final_X_train = pd.concat([ScaledN_X_train, NomEncoded_X_train, BinImputedC_X_train, Parsed_max_power_train, Parsed_max_torque_train], axis=1)
Final_X_val = pd.concat([ScaledN_X_val, NomEncoded_X_val, BinImputedC_X_val, Parsed_max_power_val, Parsed_max_torque_val], axis=1)
Final_X_train, Final_X_val = Final_X_train.align(Final_X_val, join='left', axis=1, fill_value=0)
```

`Pd.concat()` concatenates the features and takes the features you want to combine into one dataframe as an argument.

The last line is there to make sure that the training set and the validation set both have the same set of features.

Feature Selection:

Feature selection is the process of choosing a subset of the most relevant features (variables or predictors) from a larger set for use in a model or analysis. It aims to improve model performance, reduce computational costs, and enhance model interpretability.

Now, To make the model more focused and efficient, I used something called feature selection. This means picking the most important columns (or "features") from the dataset and ignoring the ones that are not very helpful.

I used a tool called `SelectKBest`. `SelectKBest` helps choose the top `k` features that are most relevant to the prediction (the thing we want to predict). In this case, I chose the top 22 features.

What is `chi2`? To decide which features are most important, I used a statistical method called chi-squared. It checks how strongly each feature is related to the target. This method is commonly used when the data contains only positive values.

```
selectK=SelectKBest(score_func=chi2,k=22)
SKX_train=selectK.fit_transform(Final_X_train,y_train)
SKX_val=selectK.transform(Final_X_val)
```

only the top 22 most useful columns, based on how well they relate to the class feature. This helps the machine learning models perform better and faster.

```

rf_model = RandomForestClassifier(n_estimators=100, random_state=0)
rf_model.fit(Final_X_train, y_train)
selector = SelectFromModel(rf_model, threshold='median', prefit=True)
X_train_rfMedian = selector.fit_transform(Final_X_train,y_train)
X_val_rfMedian = selector.transform(Final_X_val)
MedianSFMRFselected_features = Final_X_train.columns[selector.get_support()]

```

In this part, I used another way to select the most important features from the dataset. This time, instead of using a statistical test like chi-squared, I used a machine learning model itself to find out which features are most useful. I used a RandomForestClassifier. A random forest is a popular machine learning algorithm that works well for both classification and regression tasks. It is made up of many decision trees, and it can also tell us how important each feature is. After training the random forest model on the training data, I used a tool called SelectFromModel. This tool looks at the feature importances learned by the model and selects only the most important ones. In this case, I told SelectFromModel to keep only the features that had an importance greater than or equal to the median. That means it kept the top 50% of features based on importance.

This method used the random forest itself to decide which features were most helpful.

```

xg_model = XGBClassifier(n_estimators=100, random_state=0)
xg_model.fit(Final_X_train, y_train)
selector = SelectFromModel(xg_model, threshold='mean', prefit=True)
X_train_xgMean = selector.fit_transform(Final_X_train,y_train)
X_val_xgMean = selector.transform(Final_X_val)
SFMselected_features = Final_X_train.columns[selector.get_support()]

```

Now before I start explaining this step let me first define the XGBoost

XGBoost (short for Extreme Gradient Boosting) is a powerful and fast machine learning algorithm that often gives great results, especially in competitions. Like RandomForest, it can also show how important each feature is after training. Both XG and RandomForest are ensemble learning methods, but what is that exactly? Ensemble learning is a technique where multiple models (usually called "weak learners") are combined to make better predictions than any single model

alone. It's like asking a group of people for their opinions and taking a vote or average—this usually gives better decisions than asking just one person.

Now back to the code. In this step, I used another method to choose the most important features, but this time with a different model called XGBoost. I used a threshold of 'mean'. That means it kept only the features that had an importance score higher than the average. This is another way to shrink the dataset to only the most helpful features

Balancing The Dataset:

Sometimes in machine learning, the data is imbalanced, meaning there are way more examples of one class than the other (for example, way more healthy patients than sick ones). This can make models biased—they might just guess the majority class all the time.

I used three tools to get multiple results:

SMOTE: creates new, fake but realistic samples of the minority class by copying and slightly altering existing ones. This helps the model learn better about the minority class(Over Sampling).

Tomek Links: a cleaning method. It removes data points that are very close to each other but belong to different classes. These are usually near the decision boundary and may cause confusion for the model(Under Sampling).

SMOTETomek: This is a combination of SMOTE and Tomek Links. First, it creates new samples (like SMOTE), and then it cleans the data (like Tomek Links). It's often a good all-in-one solution for dealing with imbalanced data.


```

smtk = SMOTETomek()
tk=TomekLinks()
sm=SMOTE()

X_res_rfMedian, y_res_rfMedian = smtk.fit_resample(X_train_rfMedian, y_train)

X_tkrf, y_tkrf = tk.fit_resample(X_train_rfMedian, y_train)

X_tkxg, y_tkxg = tk.fit_resample(X_train_xgMean, y_train)

X_res_xgMean, y_res_xgMean = smtk.fit_resample(X_train_xgMean, y_train)

X_SMOTErF, y_SMOTErF = sm.fit_resample(X_train_rfMedian, y_train)

X_SMOTExg, y_SMOTExg = sm.fit_resample(X_train_xgMean, y_train)

X_tkSK, y_tkSK = tk.fit_resample(SKX_train, y_train)

X_res_SK, y_res_SK = smtk.fit_resample(SKX_train, y_train)

X_SMOTESK, y_SMOTESK = sm.fit_resample(SKX_train, y_train)

```

The first three lines are used to create objects of each rebalancing tool, the rest of the code is to apply each tool on each subset of features that we selected when we applied the feature selection techniques. Take a look at the following table..

Dataset Name	Feature Selection used	Rebalancing method
x_res_rfMedian	RandomForest	SMOTETomek
x_tkrf	RandomForest	TomekLinks
x_SMOTErF	RandomForest	SMOTE
x_res_xgMean	XGBoost	SMOTETomek
x_tkxg	XGBoost	TomekLinks
x_SMOTExg	XGBoost	SMOTE
x_res_SK	SelectKBest	SMOTETomek
x_tkSK	SelectKBest	TomekLinks
x_SMOTESK	SelectKBest	SMOTE

These datasets were all created to test which **feature selection + rebalancing** combination works best when we get to training the models.

Training & Evaluation:

After preparing all the datasets with different feature selection and balancing techniques, I trained and tested three machine learning models:

```
Final_RandomForest= RandomForestClassifier(random_state=0,class_weight='balanced')
Final_XGC= XGBClassifier()
Final_MLP= MLPClassifier(max_iter=250, random_state=0)
models = [Final_RandomForest,Final_XGC,Final_MLP]
datasets = {
    "RF_Median_smtk": (X_res_rfMedian, y_res_rfMedian, X_val_rfMedian, y_val),
    "XG_Mean_smtk": (X_res_xgMean, y_res_xgMean, X_val_xgMean, y_val),
    "RF_Median_tk":(X_tkrf, y_tkrf ,X_val_rfMedian, y_val),
    "XG_Mean_tk":(X_tkxg, y_tkxg ,X_val_xgMean, y_val),
    "RF_Median_SMOTE":(X_SMOTErF, y_SMOTErF,X_val_rfMedian, y_val),
    "XG_Mean_SMOTE":(X_SMOTExg, y_SMOTExg,X_val_xgMean, y_val),
    "SK_smtk": (X_res_SK, y_res_SK , SKX_val, y_val),
    "SK_tk": (X_tkSK, y_tkSK, SKX_val, y_val),
    "SK_SMOTE": (X_SMOTESK, y_SMOTESK , SKX_val, y_val),
}
```

Random Forest (RF): This is a tree-based model that uses many decision trees to make better predictions. I used it with a setting that tries to balance the class distribution by giving more weight to the minority class.

XGBoost (XGC): A powerful boosting model that builds trees one by one and tries to fix the previous one's mistakes. It's known for great performance in many competitions.

MLP (Multi-Layer Perceptron): A simple neural network model with multiple layers that learns from the data through iterations. I ran each of these models on every dataset variation I made before, like "RF_Median_smtk" or "SK_SMOTE". These names just describe how the features were selected and which balancing method was used.

```

Final_RandomForest= RandomForestClassifier(random_state=0,class_weight='balanced')
Final_XGC= XGBClassifier()
Final_MLP= MLPClassifier(max_iter=250, random_state=0)
models = [Final_RandomForest,Final_XGC,Final_MLP]
datasets = {
    "RF_Median_smtk": (X_res_rfMedian, y_res_rfMedian, X_val_rfMedian, y_val),
    "XG_Mean_smtk": (X_res_xgMean, y_res_xgMean, X_val_xgMean, y_val),
    "RF_Median_tk":(X_tkrf, y_tkrf ,X_val_rfMedian, y_val),
    "XG_Mean_tk":(X_tkxg, y_tkxg ,X_val_xgMean, y_val),
    "RF_Median_SMOTE":(X_SMOTErF, y_SMOTErF,X_val_rfMedian, y_val),
    "XG_Mean_SMOTE":(X_SMOTExg, y_SMOTExg,X_val_xgMean, y_val),
    "SK_smtk": (X_res_SK, y_res_SK , SKX_val, y_val),
    "SK_tk": (X_tkSK, y_tkSK, SKX_val, y_val),
    "SK_SMOTE": (X_SMOTESK, y_SMOTESK , SKX_val, y_val),
}
results = []
for dataset_name, (X_train, y_train, X_val, y_val) in datasets.items():
    for model in models:
        model.fit(X_train, y_train)
        y_pred = model.predict(X_val)
        confusion = confusion_matrix(y_val, y_pred)
        class_report = classification_report(y_val, y_pred, output_dict=True)
        results.append({
            "Dataset": dataset_name,
            "Model": model.__class__.__name__,
            "Accuracy": class_report["accuracy"],
            "Precision (weighted)": class_report["weighted avg"]["precision"],
            "Recall (weighted)": class_report["weighted avg"]["recall"],
            "F1-Score (weighted)": class_report["weighted avg"]["f1-score"],
            "TN": confusion[0, 0],
            "FP": confusion[0, 1],
            "FN": confusion[1, 0],
            "TP": confusion[1, 1],
        })

df_results = pd.DataFrame(results)

```

For each model and dataset, I Trained the model using the training data. Then made predictions using the validation set, and checked the predictions against the actual values using evaluation metrics...

Accuracy: How many predictions were correct.

Precision: How good the model is at not raising false alarms.

Recall: How well it catches all the actual positives.

F1-score: A balance between precision and recall.

Also saved the **confusion matrix** values: True Negatives (TN), False Positives (FP), False Negatives (FN), and True Positives (TP), which give a deeper look at what the model got right and wrong.

```
df_results.to_csv('results.csv', index=False)
```

In the end, I stored all the results in a table called `df_results`. This table lets me compare how each model performed on each dataset version in a clean and organized way.

Error Analysis:

Based on the results, the `RandomForestClassifier` models trained on the `RF_Median_tk` and `XG_Mean_tk` datasets emerge as the most balanced and reliable models. These models consistently achieved high accuracy scores of approximately 0.930 and above, coupled with relatively low false positive rates (around 0.009) and acceptable false negative rates (approximately 0.969 to 0.971). Such metrics indicate that these models effectively balance between correctly identifying both positive and negative cases, demonstrating a lower degree of bias. In contrast, `XGBClassifier` models trained on the same datasets exhibited even higher accuracy (up to 0.936) and extremely low false positive rates (sometimes even 0), but at the cost of very high false negative rates (close to 0.999). This suggests a substantial bias towards negative predictions, potentially undermining the model's

performance in detecting positive cases. Overall, `RandomForestClassifier` models are recommended as they combine high accuracy with a more equitable trade-off between false positives and false negatives, minimizing classification bias and enhancing model robustness.

For more details check out the `df_results` after running the code

Conclusion:

This comprehensive analysis demonstrates a robust approach to vehicle insurance fraud detection, leveraging various data mining and machine learning techniques. The systematic process, starting from data exploration and handling of missing values, laid a solid foundation for model development. The evaluation of Random Forest, XGBoost, and MLP across these prepared datasets provides a clear framework for identifying the most effective combination of preprocessing steps and predictive models, ultimately leading to more accurate and reliable fraud detection systems for insurance providers.

Notes:

Everything used in this project is based on Kaggle's mini-course, Data mining and machine learning course from Al-Ahliyya Amman University,

And lastly [sci-kit-learn.org](https://scikit-learn.org) all of that to make an honest model with no bias , no overfitting or underfitting and certainly no data leakage.

The code might take some time to run, sorry about that :).